

Auto-ABM: A Simulation-First Workbench for Trace-Grounded Agent-Based Modeling

Anonymous ACL submission

Abstract

Agent-based modeling (ABM) studies how system-level patterns emerge from the local decisions of interacting agents. As ABM has become a central method for computational social science, epidemiology, economics, and complex-systems research, its practical burden has also grown: researchers spend substantial time turning hypotheses into executable models, running and comparing simulations, tracing mechanisms, and packaging reproducible evidence. Large language models can accelerate programming, yet many ABM workflows still leave assumptions, traces, explanations, experiments, and exports outside the same research loop. We present Auto-ABM, to our knowledge the first AI-native, simulation-first workbench for ABM research. Auto-ABM organizes specialized research agents around a versioned simulation object backed by a deterministic Mesa kernel and a trace ledger. It supports mechanism proposal, model formalization, execution, trace-grounded explanation, adaptive experimentation, and ODD-aligned export in one workflow. Across a planned benchmark of eight ABM task families, draft pilot estimates indicate higher averaged scores for executability, mechanism validity, evidence grounding, experimental design, and reproducible reporting than code-centered baselines.

1 Introduction

Agent-based modeling is a powerful methodology for studying systems in which macro-level patterns emerge from local interactions among heterogeneous agents (Bonabeau, 2002; Macal and North, 2010). Its appeal has increased as researchers seek models that can express adaptation, spatial structure, network effects, and policy interventions without collapsing them into a single closed-form equation. Yet ABM remains labor-intensive. A typical study must turn a research question into mechanisms, encode those mechanisms in a simulator, validate assumptions, compare runs, interpret

traces, and document the model in a form that another researcher can inspect.

Recent tools can generate simulator code or assist with a particular modeling environment. This lowers the programming barrier, but it rarely changes the organization of the research object itself. The executable script still becomes the center of gravity. Traces, ODD descriptions, explanations, experiment plans, and reproduction bundles are often assembled after the run, which makes it difficult to audit how a claim about emergent behavior was produced.

Auto-ABM starts from a different premise. In ABM research, the durable object is not a script but a versioned simulation: a bundle of assumptions, parameters, runs, traces, explanations, experiments, and exportable evidence. The system treats language models as research collaborators whose actions are constrained by this object. A hypothesis agent proposes candidate mechanisms, a modeling agent formalizes the selected candidate, an execution agent delegates runs to a deterministic kernel, an evidence agent reads trace intervals, an experiment agent designs sweeps or interventions, and a documentation agent maintains ODD-aligned descriptions. These roles cooperate through the trace ledger, so explanatory language is downstream of executable evidence.

This paper contributes:

- a simulation-first workflow for LLM-assisted ABM that keeps model structure, execution traces, explanations, experiments, and export artifacts linked throughout a research session;
- a multi-agent orchestration protocol in which specialized research roles coordinate through versioned simulation objects rather than through arbitrary code edits;
- a trace-grounded explanation mechanism that separates deterministic analytics from natural-

language narration and verifies cited facts against the selected run interval;

- an AI-native software system that integrates mechanism proposal, model execution, explanation, experimentation, and ODD-aligned export inside one ABM research workbench.

2 Related Work

ABM has long been used to study complex adaptive systems by specifying local rules and observing emergent population-level behavior (Bonabeau, 2002; Macal and North, 2010). Classic families such as segregation, epidemic spread, threshold cascades, innovation diffusion, opinion dynamics, cooperation, and forest-fire spread provide reusable testbeds for modeling assumptions (Schelling, 1971; Kermack and McKendrick, 1927; Granovetter, 1978; Bass, 1969; Deffuant et al., 2000; Nowak, 2006; Drossel and Schwabl, 1992). Toolkits such as NetLogo (Wilensky, 1999) and Mesa (Kazil et al., 2020; ter Hoeven et al., 2025) make these models executable, while the ODD protocol standardizes model description and supports replication (Grimm et al., 2020). Auto-ABM builds on this tradition by making the simulation object, not the surrounding code, the unit that the conversational system creates, revises, runs, and exports.

Recent systems use LLMs to assist simulation authoring, including NetLogo-oriented assistants and verifier-assisted ABM code generation (Chen et al., 2024; Martínez et al., 2024; Niu et al., 2024). These systems show that language models can help users translate natural-language scenarios into executable models, but they generally focus on model construction rather than on the full evidence lifecycle of ABM research. A broader line of work studies agentic language-model systems that interleave reasoning and tool use (Yao et al., 2023). Auto-ABM adopts this general pattern, but constrains it to ABM-specific research roles and evidence objects. Its central distinction is that generated explanations, charts, and experiments are resolved against recorded simulation traces, providing a stronger provenance boundary than free-form code generation alone.

3 System Overview

Auto-ABM is a desktop research workbench for ABM. Its central abstraction is a versioned simulation rather than a generated program. A project contains one or more simulations, and each simulation

binds the model specification, ODD document, interface state, run records, traces, experiment plans, explanations, and export artifacts. The abstraction follows the ABM view that macro patterns arise from local agent rules (Bonabeau, 2002; Macal and North, 2010), while making intermediate evidence inspectable during routine modeling.

3.1 Design Principles

Four principles shape the system. First, Auto-ABM is simulation-centered: the user creates, revises, runs, and exports simulations, while code remains an implementation detail. Second, the trace is the source of truth. Metrics, events, mechanism firings, and spatial snapshots are written as run evidence and reused by charts, explanations, and export. Third, the executable core is deterministic; a run is identified by its model version, seed, parameter snapshot, and kernel version. Fourth, language generation is evidence-constrained. The LLM may summarize or narrate, but cited ticks, metrics, mechanisms, and events must appear in the trace slice supplied by the system.

The system also uses the ODD protocol as a documentation spine (Grimm et al., 2020). An ODD document is derived from the model configuration and remains linked to the model version. When a user changes model structure, Auto-ABM updates the ODD draft and preserves user-authored text when possible. This makes model description a live part of the workflow rather than a final write-up step.

3.2 Architecture

Auto-ABM has three conceptual layers. The interaction layer presents conversation, simulation views, trace explanation, experiment controls, and ODD documentation. The coordination layer maintains versioned project state, mediates role-specific actions, and records provenance for runs and exports. The kernel layer is a Python package built on Mesa (Kazil et al., 2020; ter Hoeven et al., 2025). It executes formal model specifications, emits trace records, derives mechanism graphs, and evaluates batch experiments. This separation keeps the interface responsive without allowing visualization state to become the source of model behavior.

3.3 End-to-End Data Flow

A typical session begins with a research question. The hypothesis agent proposes candidate mechanisms, each paired with expected macro behavior

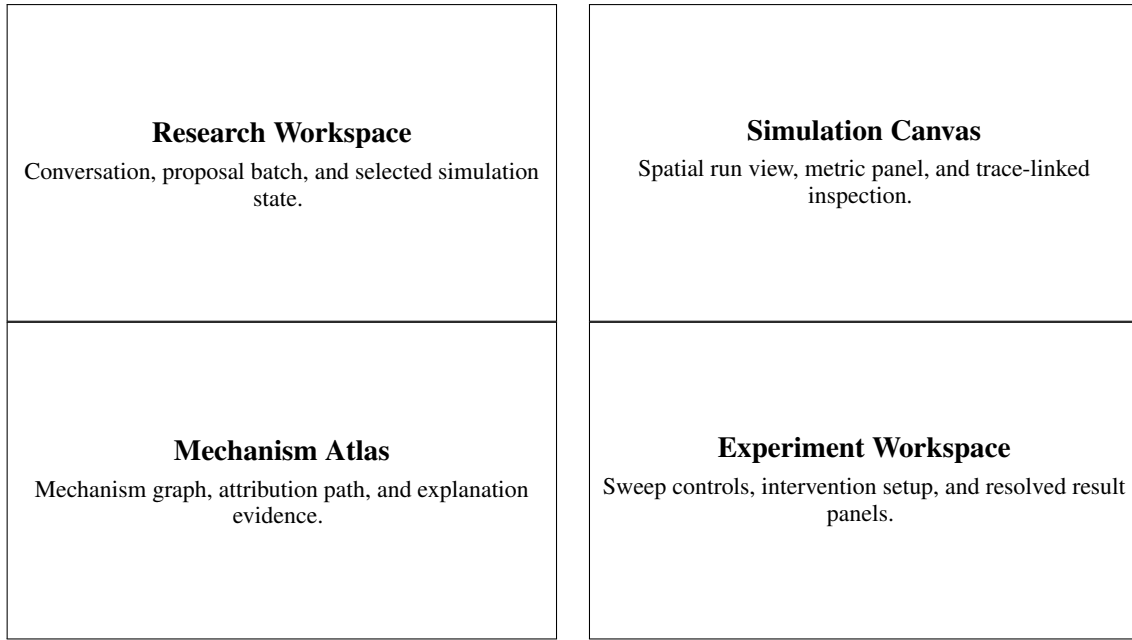


Figure 1: System overview placeholder. The camera-ready version will replace these boxes with four screenshots from the current Auto-ABM workbench, covering the research workspace, simulation canvas, mechanism atlas, and experiment workspace.

and an ODD-oriented rationale. After the user selects a candidate, the modeling agent formalizes it as a simulation specification and the execution agent records a versioned run. The user can then change parameters, inspect agents, select trace intervals, request explanations, configure experiments, and export a reproduction package. Figure 2 shows this shared path across forward and inverse research use cases.

Forward and inverse research paths share the same trace layer. In a forward path, the user starts from a mechanism hypothesis and inspects the resulting macro behavior. In an inverse path, the user starts from a target phenomenon and asks the system to propose candidate mechanisms. In both cases, claims about model behavior are grounded in real runs and trace records, not in free-form model output.

4 Methods

4.1 Research Lifecycle at a Glance

The current system implements a six-stage lifecycle from mechanism drafting to reproducible export. The key design choice is to assign each phase to a research role with a restricted evidential responsibility. The roles cooperate through persistent simulation objects rather than through unstructured edits to source code. Appendix B provides the full lifecycle table.

4.2 Simulation-Centered Multi-Agent Orchestration

Role specialization. Auto-ABM follows the agentic pattern of interleaving reasoning with external action (Yao et al., 2023), but its action space is organized around ABM research roles. The hypothesis agent expands the search space by proposing alternative mechanism families for the same phenomenon. The modeling agent narrows this space by converting an adopted proposal into an explicit specification with agent types, state variables, environment assumptions, mechanisms, observables, and initial conditions. The execution agent submits the specification to the deterministic kernel and records version, seed, parameter snapshot, and run provenance.

Evidence handoff. Downstream roles operate only after a run exists. The evidence agent transforms a selected trace interval into mechanism activity, state-transition attribution, changepoints, and counterfactual comparisons. The language narrator receives this structured evidence packet, not an open-ended invitation to invent measurements. The experiment agent designs parameter sweeps or scheduled interventions and binds result views to stored run records. Finally, the documentation agent updates the ODD narrative and prepares a reproduction package. The same trace interval can therefore appear in the chart, mechanism atlas, at-

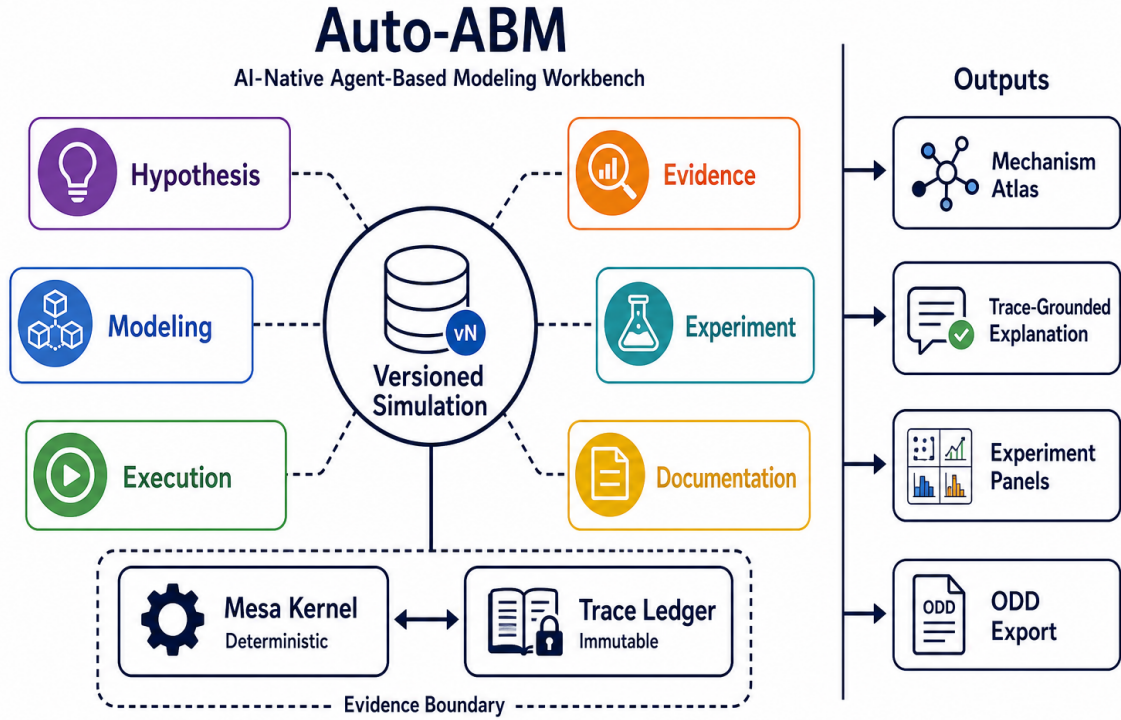


Figure 2: Method architecture of Auto-ABM. Specialized research agents operate over a shared versioned simulation object. The deterministic Mesa kernel and trace ledger define the evidence boundary for mechanism visualization, explanation, experiment analysis, ODD updates, and reproducible export.

tribution view, and explanation text without duplicating the evidential source.

The workbench supports three operating modes. Research mode permits modeling and experiments while requiring confirmation for structural model edits, large batches, overwrites, and export operations. Dialogue mode is read-only: the agents may inspect existing simulations and explain evidence, but they cannot mutate the research object. Autonomous exploration mode chains proposal, short trial runs, ranking, and reporting for unattended exploration; the evaluation in this paper focuses on research mode because it is the end-to-end workflow most directly comparable to existing modeling assistance.

Candidate simulations are generated as diverse mechanism drafts and then mapped to executable families. The current library includes epidemic diffusion, segregation, wildfire, bounded-confidence opinion, innovation diffusion, public-goods cooperation, threshold influence, and platform rumor moderation. Each specification is declarative: it states the agents, state variables, environment parameters, mechanisms, observables, initialization assumptions, and interface defaults. As a result,

model assumptions remain inspectable even when they are proposed through natural language.

4.3 Mechanism Atlas and Trace-Grounded Explanation

The most important methodological choice is to split explanation into two layers. The first layer is deterministic. Given a model configuration, the kernel derives a mechanism graph whose nodes represent agent types, state variables, parameters, mechanisms, and observers. Its edges correspond to explicit structural links or literal references in the configuration. A graph edge is therefore a record of model structure rather than an LLM-generated causal claim.

The second layer reads the run trace. A trace contains metric observations, events, mechanism firings, and, when requested, space snapshots. The coordination layer computes three kinds of trace analytics. Mechanism activity counts which mechanisms fired during an interval. Attribution maps an observed metric to an agent state and decomposes the metric change into signed state-transition flows by mechanism. Change point detection scans metric series for salient slope changes. Counterfactual

runs reuse the same model version and seed while changing parameters or scheduled interventions, then compare trajectories.

The language model receives only a structured explanation context built from the selected trace interval, related ODD passages, and mechanism references. After generation, a citation verifier checks every cited tick, metric value, event name, and mechanism reference against the context. Unsupported citations are rejected or marked as speculative. The resulting explanation is therefore not merely a natural-language summary. It is a view over trace evidence, linked back to the run.

4.4 Adaptive Deep-Experiment Workspace

ABM analysis often moves from a single run to parameter sweeps, interventions, and robustness checks. Auto-ABM supports this transition by letting the experiment agent synthesize a declarative experiment view. The view specifies controls, sweep variables, chart intents, and preferred metrics. The interaction layer renders this specification as manipulable controls and result panels, while the coordination layer retains responsibility for execution and data binding.

This design avoids a common failure mode of LLM-generated analysis. The experiment agent does not provide chart data. Instead, it provides a declarative view request that names the source run or experiment and the requested fields. The coordination layer resolves the request against real run records and rejects missing fields. Batch experiments are expanded into run plans by the kernel, executed with controlled seeds, and persisted as run records under the same simulation. This lets a user move from a question such as “sweep debunking start time” to a concrete experiment panel without breaking the provenance chain.

4.5 Integrity Mechanisms

The system includes several integrity gates. The specification gate checks model shape, observable availability, parameter ranges, and ODD consistency before or after edits. The explanation gate validates citations against trace evidence. The visualization gate resolves chart requests against stored run records and rejects fabricated columns. The mode gate prevents state mutation in dialogue mode. The approval gate asks before structural model edits, large batch runs, destructive overwrites, and reproduction export. These checks are deliberately small and explicit; they are easier to

audit than a single broad instruction telling a language model to be faithful.

5 Experiments

The final experiments will measure whether Auto-ABM improves the quality, explainability, and reproducibility of LLM-assisted ABM work. Since the controlled study has not yet been run, the numeric values in this section are draft estimates used to fix the experimental design and figure layout. They must be replaced with measured results before submission.

5.1 Tasks and Systems

We use eight ABM tasks that cover common model families and visual forms: Schelling segregation (Schelling, 1971), SIR spread (Kermack and McKendrick, 1927), platform rumor moderation, Granovetter-style threshold cascades (Granovetter, 1978), Bass-style innovation diffusion (Bass, 1969), bounded-confidence opinion dynamics (Deffuant et al., 2000), public-goods cooperation (Nowak, 2006), and forest-fire spread (Drossel and Schwabl, 1992). Each task asks a workflow to instantiate a runnable model, produce at least one run, explain a selected interval, run a sweep or intervention experiment, and export a reproduction artifact. The full protocol schematic is included in Appendix A.

We compare four workflows. *Direct Python* asks the LLM to write a single runnable Python/Mesa script from the task prompt. *NetLogo generation* asks the LLM to generate a NetLogo-style model and run instructions, reflecting the language and ecosystem used by NetLogo and NetLogo-assistance work (Wilensky, 1999; Chen et al., 2024; Martínez et al., 2024). *Harness* uses a general coding-agent loop with tests and command execution, close to verifier-assisted code generation (Niu et al., 2024). *Auto-ABM* uses the system described above. Each workflow is run with three model backends: Claude Opus 4.6, DeepSeek-V4, and the latest Qwen model available for the final run.

5.2 Metrics

All metrics are reported as scores on a five-point scale. For judge-based dimensions, we use an ensemble of three blinded LLM judges following recent work on LLM-as-judge evaluation (Liu et al., 2023; Zheng et al., 2023). The judge prompt includes the task, model description, run summaries, sampled traces, exported files, and the system explanation when one is available. We report the

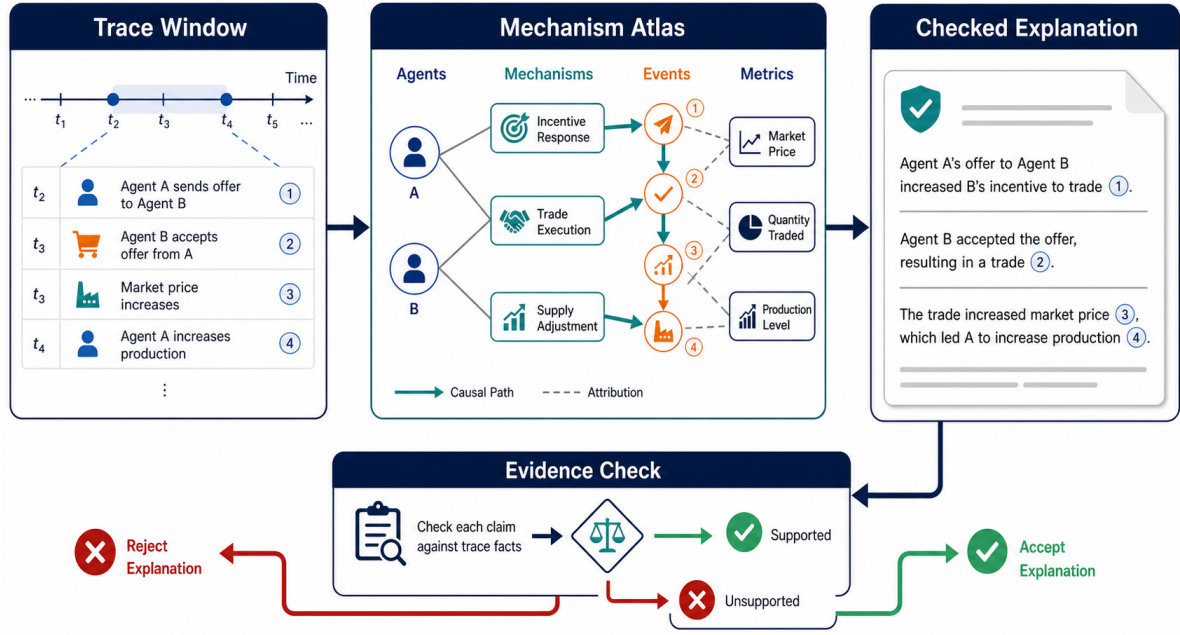


Figure 3: Mechanism atlas and trace-grounded explanation. The system links a selected trace window to mechanism paths, attribution evidence, and checked explanatory statements. Unsupported claims are rejected by the evidence check.

mean score across judges, tasks, and model back-ends.

- **Executable validity.** A rubric score reflects whether the workflow produces a runnable model, a completed run, and the required downstream artifacts.
- **Model quality.** LLM judges score whether the model structure, assumptions, and observables match the scientific intent of the task.
- **Mechanism validity.** LLM judges score whether the proposed mechanisms are plausible for the target phenomenon and are expressed in the executable model.
- **Evidence grounding.** A trace verifier scores cited ticks, metrics, events, and mechanisms, with judge review for whether the explanation uses those facts appropriately.
- **Experiment quality.** LLM judges score whether sweeps or interventions are meaningful, executable, and tied to the research question.
- **Reproduction completeness.** A checklist score measures whether the export includes the model specification, ODD document, run metadata, seeds, traces, experiment plans, and provenance manifest.

The overall score reported below is the un-weighted mean of these six dimensions.

5.3 Main Results

Metric	Direct	NetLogo	Harness	Auto-ABM
Exec. validity	2.9	3.2	3.9	4.7
Model quality	3.1	3.3	3.6	4.4
Mechanism val.	2.8	3.1	3.4	4.5
Evidence ground.	2.3	2.5	3.1	4.6
Experiment qual.	2.5	2.7	3.4	4.5
Repro. complete	2.1	2.4	3.2	4.6
Overall	2.62	2.87	3.43	4.55

Table 1: Draft score estimates on a five-point scale, averaged over eight tasks and three model backends. These values are placeholders for the planned experiment. The final paper must replace them with measured results.

The expected pattern in Table 1 is that direct generation often produces plausible code but receives low scores for evidence grounding and reproduction completeness. NetLogo generation improves alignment with ABM conventions, but still separates model construction from trace-backed explanation and experiment reporting. The harness baseline improves executability because it can iterate on failures, yet its outputs remain code-centered. Auto-ABM is expected to score higher across the evidence-facing dimensions because the kernel, trace, ODD document, experiment runner, and export manifest are shared across all tasks. Figure 4 summarizes the same draft estimates as a multi-panel score comparison.

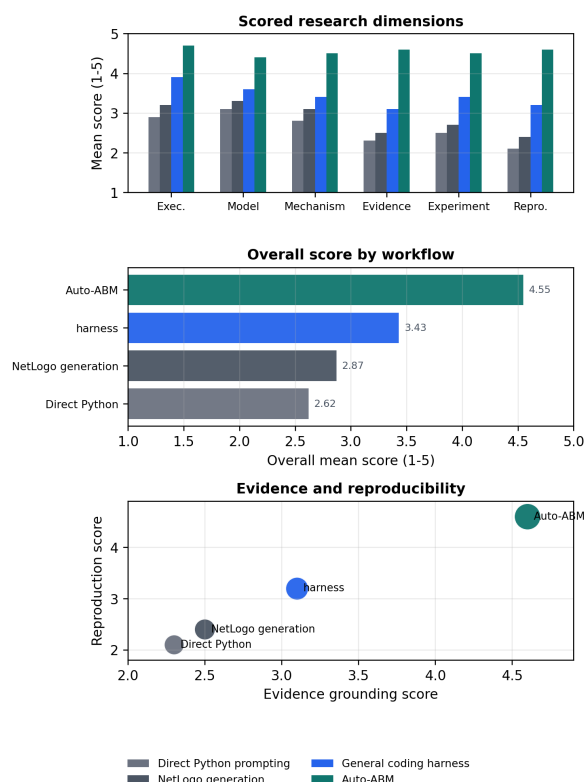


Figure 4: Draft result dashboard derived from Table 1. The panels compare score dimensions, overall averages, and the relationship between evidence grounding and reproduction completeness. Values are placeholders pending the controlled evaluation.

6 Conclusion

Auto-ABM reframes LLM-assisted ABM as a simulation-first research workflow. Rather than treating code generation as the endpoint, it keeps simulations, runs, traces, ODD documents, experiments, and exports as linked research objects. Its main contribution is the combination of specialized research agents, a deterministic Mesa kernel, a specification-derived mechanism atlas, evidence-checked explanations, and adaptive experiment views. The result is a workbench in which researchers can move from a question to a runnable model, inspect the evidence behind emergent behavior, and export a reproducible artifact from the same interface.

Limitations

This draft describes a system that is still being prepared for final submission. The controlled evaluation is planned but not yet run, and the numeric results in Section 5 are placeholders. The current kernel covers common ABM families, but it is not a general simulator for every domain. The mech-

anism graph is derived from explicit specification references, so it may miss implicit dependencies hidden inside custom behavior implementations. Evidence validation checks cited trace facts, but it does not prove that the narrative interpretation is the only valid explanation. Autonomous exploration is implemented as a product direction and partial workflow, but the main evaluation should focus on the research-mode loop that is already end-to-end.

Ethics Statement

Auto-ABM is intended to support modeling and interpretation, not to certify policy conclusions. ABM results depend on assumptions about agents, environments, and mechanisms. The workbench makes these assumptions more visible through ODD documents, trace evidence, and reproducible export, but users remain responsible for domain validation. Generated explanations should be treated as analytic summaries linked to trace evidence, not as causal proof outside the modeled system.

References

- Frank M. Bass. 1969. [A new product growth for model consumer durables](#). *Management Science*, 15(5):215–227.
- Eric Bonabeau. 2002. [Agent-based modeling: Methods and techniques for simulating human systems](#). *Proceedings of the National Academy of Sciences*, 99(suppl 3):7280–7287.
- John Chen, Xi Lu, Yuzhou Du, Michael Rejtig, Ruth Bagley, Michael Horn, and Uri Wilensky. 2024. [Learning agent-based modeling with LLM companions: Experiences of novices and experts using ChatGPT and NetLogo Chat](#). In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery.
- Guillaume Deffuant, David Neau, Frederic Amblard, and Gerard Weisbuch. 2000. [Mixing beliefs among interacting agents](#). *Advances in Complex Systems*, 3(1–4):87–98.
- Barbara Drossel and Franz Schwabl. 1992. [Self-organized critical forest-fire model](#). *Physical Review Letters*, 69(11):1629–1632.
- Mark Granovetter. 1978. [Threshold models of collective behavior](#). *American Journal of Sociology*, 83(6):1420–1443.
- Volker Grimm, Steven F. Railsback, Christian E. Vincenot, Uta Berger, Cara Gallagher, Donald L. DeAngelis, Bruce Edmonds, Jun Ge, Jarl Giske, Jan Groeneweld, Alice S. A. Johnston, Andreas Milles, Jacob

503	Nabe-Nielsen, J. Gary Polhill, Viktoriia Radchuk, Marie-Sophie Rohwäder, Richard A. Stillman, Jan C. Thiele, and Daniel Ayllón. 2020. The ODD protocol for describing agent-based and other simulation models: A second update to improve clarity, replication, and structural realism . <i>Journal of Artificial Societies and Social Simulation</i> , 23(2):7.	558
504		559
505		560
506		561
507		562
508		563
509		
510	Jackie Kazil, David Masad, and Andrew Crooks. 2020. Utilizing Python for agent-based modeling: The Mesa framework . In <i>Social, Cultural, and Behavioral Modeling</i> , volume 12268 of <i>Lecture Notes in Computer Science</i> , pages 308–317. Springer.	
511		
512		
513		
514		
515	William Ogilvy Kermack and Anderson G. McKendrick. 1927. A contribution to the mathematical theory of epidemics . <i>Proceedings of the Royal Society of London. Series A, Containing Papers of a Mathematical and Physical Character</i> , 115(772):700–721.	
516		
517		
518		
519		
520	Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-Eval: NLG evaluation using GPT-4 with better human alignment . In <i>Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing</i> , pages 2511–2522.	
521		
522		
523		
524		
525		
526	Charles M. Macal and Michael J. North. 2010. Tutorial on agent-based modelling and simulation . <i>Journal of Simulation</i> , 4(3):151–162.	
527		
528		
529	Joseph Martínez, Brian Llinás, Jhon G. Botello, Jose J. Padilla, and Erika F. Frydenlund. 2024. Enhancing GPT-3.5’s proficiency in NetLogo through few-shot prompting and retrieval-augmented generation . In <i>Proceedings of the 2024 Winter Simulation Conference</i> , pages 666–677.	
530		
531		
532		
533		
534		
535	Tong Niu, Weihao Zhang, and Rong Zhao. 2024. Solution-oriented agent-based models generation with verifier-assisted iterative in-context learning . In <i>Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems</i> , pages 1473–1481. ArXiv:2402.02388.	
536		
537		
538		
539		
540		
541	Martin A. Nowak. 2006. Five rules for the evolution of cooperation . <i>Science</i> , 314(5805):1560–1563.	
542		
543	Thomas C. Schelling. 1971. Dynamic models of segregation . <i>Journal of Mathematical Sociology</i> , 1(2):143–186.	
544		
545		
546	Ewout ter Hoeven, Jan Kwakkel, Vincent Hess, Thomas Pike, Boyu Wang, and Jackie Kazil. 2025. Mesa 3: Agent-based modeling with Python in 2025 . <i>Journal of Open Source Software</i> , 10(107):7668.	
547		
548		
549		
550	Uri Wilensky. 1999. NetLogo . Center for Connected Learning and Computer-Based Modeling, Northwestern University, Evanston, IL.	
551		
552		
553	Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models . In <i>International Conference on Learning Representations</i> .	
554		
555		
556		
557		
	Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-Bench and chatbot arena . ArXiv:2306.05685.	
	A Experiment Protocol Details	564
	B Simulation Lifecycle Details	565
	C Research Role Catalogue	566
	D Trace and Export Contracts	567
	The kernel trace is an append-only record of run metadata, metric observations, events, mechanism firings, spatial snapshots, and termination status. Explanation modules read only the trace slice and record types needed for a selected interval. The export package contains the model specification, ODD document, experiment plans, run summaries, optional trace files, and a provenance manifest. The manifest records system version, kernel version, integrity checks, run identifiers, seeds, steps, parameter snapshots, model identity, and model version. This package is the object used to verify claims made about the submitted system.	568
		569
		570
		571
		572
		573
		574
		575
		576
		577
		578
		579
		580
	E Per-Backend Score Estimates	581

C. Evaluation protocol: tasks, artifacts, and scores

Task family	Model	Run	Explain	Sweep	Export	Space	Phenomenon
Schelling	●	●	●	●	●	grid	segregation
SIR spread	●	●	●	●	●	network	contagion
Rumor moderation	●	●	●	●	●	network	intervention
Threshold cascade	●	●	●	●	●	network	collective action
Innovation diffusion	●	●	●	●	●	population	adoption
Bounded opinion	●	●	●	●	●	population	polarization
Public goods	●	●	●	●	●	groups	cooperation
Forest fire	●	●	●	●	●	grid	spatial spread

Compared workflows Direct Python prompting NetLogo-style generation General coding harness Auto-ABM	Scoring dimensions executable success model and phenomenon quality evidence grounding experiment design reproduction completeness	Scoring procedure five-point rubrics LLM-judge ensemble trace verification artifact checklist
--	---	--

Figure 5: Planned experiment protocol. The task suite spans eight ABM families and five required artifacts. The same five-point scoring protocol is applied to every workflow and backend, covering executable validity, model quality, mechanism validity, evidence grounding, experiment quality, and reproduction completeness.

Stage	Coordinating role	Persistent objects	Evidence constraint
Draft	Hypothesis agent	Candidate mechanisms, expected macro patterns, ODD rationale	Candidate curves are described as hypotheses until a run is executed.
Formalize	Modeling agent	Versioned specification, parameter schema, synchronized ODD draft	Structural changes require validation and user approval.
Execute	Execution agent	Run record, metric series, spatial snapshots, trace ledger	Only the kernel may create authoritative observations.
Explain	Evidence agent and language narrator	Mechanism activity, attribution flows, cited explanation	Citations must match ticks, metrics, events, or mechanisms in the selected trace interval.
Experiment	Experiment agent	Sweep design, intervention plan, resolved result panels	Charts are bound to recorded runs rather than generated values.
Publish	Documentation agent	ODD document, reproduction package, provenance manifest	Export includes assumptions, seeds, run records, and integrity checks.

Table 2: Simulation-centered lifecycle in Auto-ABM.

Research role	May mutate state	Primary responsibility
Hypothesis agent	no	Propose mechanism families, expected macro patterns, and ODD-oriented rationales.
Modeling agent	yes	Convert an adopted hypothesis into a validated simulation specification and versioned assumptions.
Execution agent	yes	Submit the model to the deterministic kernel and preserve seed, parameter, and run provenance.
Evidence agent	no	Transform trace intervals into mechanism activity, attribution flows, changepoints, and counterfactual summaries.
Language narrator	no	Produce explanations only from evidence packets and mark unsupported statements as speculative.
Experiment agent	yes	Design sweeps and interventions whose charts are bound to stored run records.
Documentation agent	yes	Keep the ODD description, reproduction package, and provenance manifest aligned with the current simulation.

Table 3: Conceptual research roles used by Auto-ABM.

Backend	Workflow	Overall (1–5)	Model (1–5)	Evidence (1–5)	Repro (1–5)
Claude Opus 4.6	Direct Python	2.75	3.2	2.5	2.3
Claude Opus 4.6	NetLogo generation	3.00	3.4	2.7	2.6
Claude Opus 4.6	Harness	3.60	3.8	3.3	3.4
Claude Opus 4.6	Auto-ABM	4.65	4.6	4.7	4.7
DeepSeek-V4	Direct Python	2.60	3.1	2.3	2.1
DeepSeek-V4	NetLogo generation	2.85	3.3	2.5	2.4
DeepSeek-V4	Harness	3.42	3.6	3.1	3.2
DeepSeek-V4	Auto-ABM	4.54	4.4	4.6	4.6
Qwen latest	Direct Python	2.50	3.0	2.1	1.9
Qwen latest	NetLogo generation	2.75	3.2	2.3	2.2
Qwen latest	Harness	3.28	3.4	2.9	3.0
Qwen latest	Auto-ABM	4.46	4.3	4.5	4.5

Table 4: Draft per-backend score estimates. These placeholder values are included to lock table shape before the real runs.